

# Python Formulario

Nahuel Villafañe

@clasesconnahu [github.com/Fisicoloco99](https://github.com/Fisicoloco99)

**Hecho en Argentina**



5 de junio de 2026

---

*Sintaxis, funciones, módulos, proyectos, ejercicios y referencias – Edición completa*

---

Nahuel Villafañe – Python paso a paso

# Prólogo

Querido lector: este documento reúne **todo** lo que aprendiste en el curso. Desde los primeros pasos con `print("Hola mundo")` hasta la creación de ejecutables con PyInstaller, pasando por listas, diccionarios, funciones, módulos de la biblioteca estándar y proyectos completos. Todo está aquí, explicado en criollo y con ejemplos reales de tus prácticas. No lo leas como una novela, practicá cada bloque. ¡Manos a la obra!

## 1 Instalación y configuración

### ¿Qué es Python?

Python es un lenguaje interpretado, de tipado dinámico, multiplataforma y multiparadigma. Viene con su propio IDE (IDLE) y se puede usar en cualquier editor.

### Instalación en Windows

1. Descargar el instalador de [python.org](https://python.org)
2. Marcar la casilla “Add Python to PATH” (¡fundamental!)
3. Hacer clic en “Install Now”

### Verificar instalación

```
1 python --version
2 pip --version
```

### Entornos de desarrollo (IDEs) recomendados

| blue!70!black    |                                              |
|------------------|----------------------------------------------|
| IDLE             | Viene con Python, ideal para empezar         |
| VS Code          | Gratuito, extensiones, depurador integrado   |
| PyCharm          | IDE profesional (versión Community gratuita) |
| Jupyter Notebook | Excelente para ciencia de datos              |
| Thonny           | Pensado para principiantes absolutos         |

## 2 Variables y tipos de datos

### Definición

Una variable es un espacio en memoria con un nombre asociado que guarda un valor.

```
1 nombre = "Nahuel"
2 edad = 25
3 altura = 1.75
4 es_estudiante = True
```

## Reglas para nombres

- Solo letras, números y guión bajo.
- No empezar con número.
- No usar palabras reservadas (`print`, `if`, `for`, etc.).
- Convención: `snake_case`.

## Tipos principales

| blue!70!black      |                                 |
|--------------------|---------------------------------|
| <code>int</code>   | <code>entero = 42</code>        |
| <code>float</code> | <code>pi = 3.1416</code>        |
| <code>str</code>   | <code>texto = "Python"</code>   |
| <code>bool</code>  | <code>verdad = True</code>      |
| <code>list</code>  | <code>[1, 2, 3]</code>          |
| <code>tuple</code> | <code>(1, 2, 3)</code>          |
| <code>dict</code>  | <code>{"clave": "valor"}</code> |
| <code>set</code>   | <code>{1, 2, 3}</code>          |

## Conversión de tipos

```
1 int("123")      # 123
2 str(456)        # "456"
3 float("3.14")   # 3.14
4 bool(0)         # False
```

## 3 Operadores

### Aritméticos

| blue!70!black   |                 |                          |
|-----------------|-----------------|--------------------------|
| <code>+</code>  | Suma            | <code>5 + 3 = 8</code>   |
| <code>-</code>  | Resta           | <code>5 - 3 = 2</code>   |
| <code>*</code>  | Multiplicación  | <code>5 * 3 = 15</code>  |
| <code>/</code>  | División        | <code>5 / 2 = 2.5</code> |
| <code>//</code> | División entera | <code>5 // 2 = 2</code>  |
| <code>%</code>  | Módulo (resto)  | <code>5 % 2 = 1</code>   |
| <code>**</code> | Potencia        | <code>2 ** 3 = 8</code>  |

### Comparación

```
1 == != > < >= <=
```

### Lógicos

`and`, `or`, `not`

### Asignación

`=`, `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`

## 4 Entrada y salida

```
1 nombre = input(" ¿Cómo te llamas? ")
2 print("Hola", nombre)
3 print(f"Tu nombre tiene {len(nombre)} letras")
```

Parámetros útiles de print: sep (separador), end (final de línea).

## 5 Condicionales

if, elif, else

```
1 if nota >= 9:
2     print("Excelente")
3 elif nota >= 7:
4     print("Aprobado")
5 elif nota >= 5:
6     print("Regular")
7 else:
8     print("Desaprobado")
```

Operador ternario: "Par" if n%2==0 else "Impar"

## 6 Bucles

while

```
1 contador = 1
2 while contador <= 10:
3     print(contador)
4     contador += 1
```

break sale del bucle, continue salta a la siguiente iteración.

for

```
1 for i in range(1, 11):          # 1..10
2     print(i)
3
4 for letra in "Python":
5     print(letra)
6
7 for indice, valor in enumerate(lista):
8     print(indice, valor)
```

### Bucles anidados

```
1 for i in range(1, 6):
2     for j in range(1, i+1):
3         print(j, end=" ")
4     print()
```

## 7 Listas (mutables)

```
1 frutas = ["manzana", "pera", "uva"]
2 frutas.append("naranja")
3 frutas.insert(1, "kiwi")
4 frutas.remove("pera")
5 ultima = frutas.pop()
6 frutas.sort()
7 frutas.reverse()
8 indice = frutas.index("uva")
9 conteo = frutas.count("manzana")
```

## Comprensión de listas

```
1 cuadrados = [x**2 for x in range(10) if x
2             % 2 == 0]
3 matriz = [[i*j for j in range(3)] for i in
4           range(3)]
```

## 8 Tuplas (inmutables)

```
1 punto = (10, 20)
2 x, y = punto
3 tupla_un_elemento = (42,)
```

## 9 Sets (conjuntos)

```
1 A = {1,2,3}
2 B = {3,4,5}
3 union = A | B
4 interseccion = A & B
5 diferencia = A - B
6 diferencia_simetrica = A ^ B
```

## 10 Diccionarios

```
1 persona = {"nombre": "Ana", "edad": 25}
2 persona["ciudad"] = "Madrid"
3 del persona["edad"]
4 print(persona.get("nombre", "No existe"))
5 for clave, valor in persona.items():
6     print(clave, valor)
```

## 11 Cadenas (strings)

### Métodos principales

|                       |                                       |
|-----------------------|---------------------------------------|
| blue!70!black         |                                       |
| s.lower() / upper()   | Minúsculas / mayúsculas               |
| s.strip()             | Elimina espacios al inicio/final      |
| s.replace(a,b)        | Reemplaza a por b                     |
| s.split(",")          | Divide por separador                  |
| "-".join(lista)       | Une lista con guiones                 |
| s.find(sub)           | Posición de subcadena (-1 si no está) |
| s.count(sub)          | Número de apariciones                 |
| s.startswith(prefijo) | Empieza con?                          |
| s.endswith(sufijo)    | Termina con?                          |
| s.isdigit()           | ¿Solo dígitos?                        |
| s.isalpha()           | ¿Solo letras?                         |

### Formateo

```
1 nombre = "Nahuel"; edad = 25
2 print(f"Hola {nombre}, tenés {edad} años")
3 print("Hola {}, tenés {} años".format(
4     nombre, edad))
5 print("Hola %s, tenés %d años" % (nombre,
6     edad))
```

## 12 Funciones

### Definición básica

```
1 def suma(a, b):
2     """Devuelve la suma de a y b."""
3     return a + b
```

### Funciones puras vs impuras

- **Pura:** solo usa `return`, no imprime ni modifica variables externas.
- **Impura:** tiene efectos secundarios (`print`, modificar globales, etc.).

### Argumentos variables

```
1 def funcion(*args, **kwargs):
2     print(args) # tupla
3     print(kwargs) # diccionario
```

### Recursividad

```
1 def factorial(n):
2     if n <= 1:
3         return 1
4     return n * factorial(n-1)
```

## 13 Manejo de errores (excepciones)

```
1 try:
2     num = int(input("Número: "))
3     resultado = 100 / num
4 except ValueError:
5     print("Debe ingresar un número")
6 except ZeroDivisionError:
7     print("No se puede dividir por cero")
8 else:
9     print("Resultado:", resultado)
10 finally:
11     print("Siempre se ejecuta")
```

### Crear excepciones propias:

```
1 class MiError(Exception):
2     pass
3 raise MiError("Mensaje personalizado")
```

## 14 Módulos de la biblioteca estándar

### random

|                             |                            |
|-----------------------------|----------------------------|
| <code>blue!70!black</code>  |                            |
| <code>randint(a,b)</code>   | Entero entre a y b         |
| <code>random()</code>       | Float en [0,1)             |
| <code>choice(seq)</code>    | Elige un elemento al azar  |
| <code>shuffle(lista)</code> | Mezcla la lista (modifica) |
| <code>sample(seq,k)</code>  | k elementos únicos         |

### string

Constantes: `ascii_letters`, `digits`, `punctuation`, `ascii_lowercase`, `ascii_uppercase`.

### os – sistema operativo

Funciones clave:

```
1 import os
2 os.getcwd() # directorio
3 os.listdir() # lista
4 os.chdir(ruta) # cambiar
5 os.makedirs("a/b/c", exist_ok=True)
6 os.remove("archivo.txt")
7 os.rename("viejo", "nuevo")
8 os.path.exists(ruta)
9 os.path.isfile(ruta)
10 os.path.isdir(ruta)
11 os.path.getsize(ruta)
12 os.path.join("carpeta", "archivo")
13 os.system("clear") # limpia
    pantalla (Linux/Mac)
```

### shutil – operaciones de alto nivel

```
1 import shutil
2 shutil.copy(origen, destino)
3 shutil.copytree(origen, destino,
4                 dirs_exist_ok=True)
5 shutil.move(origen, destino)
6 shutil.rmtree(carpetas) # elimina
    carpeta con contenido (cuidado !)
7 shutil.make_archive("backup", "zip", "
    carpeta")
```

### pathlib – rutas modernas

```
1 from pathlib import Path
2 ruta = Path("carpeta") / "archivo.txt"
3 ruta.exists()
4 ruta.is_file()
5 contenido = ruta.read_text(encoding="utf-8")
6 ruta.write_text("texto", encoding="utf-8")
7 for archivo in Path(".").glob("*.py"):
8     print(archivo)
```

### datetime

```
1 from datetime import datetime, date,
2     timedelta
3 hoy = date.today()
4 ahora = datetime.now()
5 manana = hoy + timedelta(days=1)
6 formato = ahora.strftime("%d/%m/%Y %H:%M:%S")
7 fecha = datetime.strptime("25/12/2024", "%
8     d/%m/%Y")
```

## math

|                                          |                      |
|------------------------------------------|----------------------|
| blue!70!black                            |                      |
| pi, e                                    | Constantes           |
| sqrt(x)                                  | Raíz cuadrada        |
| ceil(x),<br>floor(x)                     | Redondeo             |
| factorial(n)                             | Factorial            |
| comb(n,k),<br>perm(n,k)                  | Combinatoria         |
| sin,<br>cos, tan,<br>radians,<br>degrees | Trigonometría        |
| hypot(x,y)                               | Distancia euclidiana |

## re – expresiones regulares

```
1 import re
2 re.search(r"\d+", texto)
3 re.findall(r"[aeiou]", texto)
4 re.sub(r"\s+", " ", texto)
5 re.split(r",", linea)
6 patron = re.compile(r"\w+@\w+\.com")
```

## json

```
1 import json
2 datos = {"nombre": "Ana", "edad": 25}
3 with open("datos.json", "w") as f:
4     json.dump(datos, f, indent=2,
5               ensure_ascii=False)
6 with open("datos.json") as f:
7     datos = json.load(f)
```

## CSV

```
1 import csv
2 with open("archivo.csv", newline="") as f:
3     reader = csv.DictReader(f)
4     for fila in reader:
5         print(fila["nombre"])
```

## collections

```
1 from collections import Counter,
2   defaultdict, deque
3 contador = Counter("abracadabra")
4 print(contador.most_common(2))
5
6 dd = defaultdict(list)
7 dd["frutas"].append("manzana")
8
9 cola = deque([1,2,3])
10 cola.appendleft(0)
11 cola.popleft()
```

## itertools

```
1 import itertools
2 list(itertools.chain([1,2], [3,4]))
3 list(itertools.combinations("ABC", 2))
4 list(itertools.permutations("ABC", 2))
5 list(itertools.product([1,2], repeat=2))
```

## sys

```
1 import sys
2 sys.argv          # argumentos de línea de
3                   # comandos
4 sys.exit(0)        # terminar programa
5 sys.path           # lista de rutas de bú
6                   # squeda de módulos
7 sys.version        # versión de Python
8 sys.platform       # sistema operativo
```

## copy

```
1 import copy
2 lista2 = copy.copy(lista1)          # copia
3                                         # superficial
4 lista2 = copy.deepcopy(lista1)      # copia
5                                         # profunda
```

## 15 Archivos (lectura/escritura)

```
1 # Escritura
2 with open("salida.txt", "w", encoding="utf
3 -8") as f:
4     f.write("contenido\n")
5
6 # Lectura
7 with open("entrada.txt", "r", encoding="
8 utf-8") as f:
9     contenido = f.read()
10    lineas = f.readlines()
```

## 16 Depuración y logging

### pdb – depurador

```
1 breakpoint()          # punto de
2                         # interrupción
3 # Comandos en (Pdb): n (next), s (step), c
4                       # (continue), p var, l (list)
```

### logging – registro profesional

```
1 import logging
2 logging.basicConfig(level=logging.DEBUG,
3                     format='%(asctime)s -
4                       %(levelname)s - %(
5 message)s',
6                     filename='app.log')
7 logging.info("Mensaje informativo")
8 logging.error("Ocurrió un error")
```

```

5     return sum(p["precio"] * p["cantidad"]
               for p in carrito)

```

## Juego del ahorcado

```

1 palabra_secreta = "python"
2 letras_acertadas = []
3 intentos = 6
4 while intentos > 0:
5     estado = "".join(letra if letra in
6                       letras_acertadas else "_" for letra
7                       in palabra_secreta)
8     print(estado)
9     letra = input("Letra: ").lower()
10    if letra in letras_acertadas:
11        continue
12    if letra in palabra_secreta:
13        letras_acertadas.append(letra)
14        if set(letras_acertadas) == set(
15            palabra_secreta):
16            print("Ganaste!")
17            break
18    else:
19        intentos -= 1
20        print(f"Te quedan {intentos} vidas")
21
22 else:
23     print(f"Perdiste, la palabra era {
24         palabra_secreta}")

```

## Generador de contraseñas

```

1 import random, string
2 def generar_contrasena(longitud=12):
3     caracteres = string.ascii_letters +
4     string.digits + string.punctuation
5     return "".join(random.choice(
6         caracteres) for _ in range(longitud))

```

## Explorador de archivos con os

Ver PRACTICA4.py: funciones mostrar\_archivos, crear\_carpetas, contar\_archivos, organizador\_archivos, etc.

## Juego de combate (Pikachu vs Meatwho)

Código completo en Clase17.py y Biblioteca.py.

## 21 Errores comunes y cómo resolverlos

|                                          |                                                    |
|------------------------------------------|----------------------------------------------------|
| blue!70!black                            |                                                    |
| SyntaxError:<br>invalid<br>syntax        | Revisar dos puntos faltantes, paréntesis, comillas |
| NameError:<br>name 'x' is<br>not defined | Variable no definida o mal escrita                 |

## 17 Buenas prácticas (PEP 8)

- Variables y funciones: `snake_case`
- Clases: `CamelCase`
- Constantes: `MAYUSCULAS`
- Indentación: 4 espacios (nunca tabs)
- Línea máxima: 79 caracteres
- Espacios alrededor de operadores: `x = y + z`
- Importaciones: agrupar (estándar, terceros, locales)
- Usar docstrings:

```
1 def funcion(param):
2     """Explicación breve."""
3     pass
```

## 18 Entornos virtuales (venv)

```
1 python -m venv mi_entorno
2 # Activar en Windows: mi_entorno\Scripts\
   activate
3 # Activar en Linux/Mac: source mi_entorno/
   bin/activate
4 pip install requests
5 pip freeze > requirements.txt
6 pip install -r requirements.txt
7 deactivate
```

## 19 PyInstaller – crear ejecutables

```
1 pip install pyinstaller
2 pyinstaller --onefile --windowed --icon=
   icono.ico mi_script.py
```

Añadir `input("Presione Enter para salir...")` al final para que la ventana no se cierre inmediatamente.

## 20 Proyectos integradores (de tus prácticas)

### Sistema de gestión de alumnos y notas

```
1 alumnos = ["Ana", "Luis", "Marta"]
2 notas = {"Ana": [8,9], "Luis": [10,7], "
   Marta": [6,8]}
3
4 def mostrar_promedios():
5     for alumno in notas:
6         promedio = sum(notas[alumno]) /
           len(notas[alumno]) if notas[
               alumno] else 0
7         print(f"{alumno}: {promedio:.2f}")
```

### Carrito de compras con diccionarios

```
1 carrito = []
2 def agregar(nombre, precio, cantidad):
3     carrito.append({"nombre": nombre, "
   precio": precio, "cantidad":
       cantidad})
4 def total():
```

|                                                                                                     |                                                                   |
|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <code>TypeError:</code><br><code>unsupported</code><br><code>operand</code><br><code>type(s)</code> | Convertir tipos (int, str) antes de operar                        |
| <code>IndexError:</code><br><code>list index</code><br><code>out of range</code>                    | Verificar <code>len(lista)</code> antes de acceder                |
| <code>KeyError</code>                                                                               | Usar <code>dict.get()</code> o comprobar <code>key in dict</code> |
| <code>ZeroDivisionError</code>                                                                      | Asegurar que el divisor no sea cero                               |
| <code>IndentationError</code>                                                                       | Revisar espacios consistentes (4 espacios)                        |

## 22 Recursos adicionales y continuación

- Documentación oficial: <https://docs.python.org/3/>
- Ejercicios: <https://exercism.org/>, <https://codewars.com/>
- Libros: *Automate the Boring Stuff with Python*, *Python Crash Course*
- Canales de YouTube: Píldoras Informáticas, Hola-Mundo, MoureDev
- Comunidad: r/learnpython en Reddit, Discord de Python

## 23 Tablas de referencia rápida

### Métodos de listas

| blue!70!black                  |                                   |
|--------------------------------|-----------------------------------|
| <code>lista.append(x)</code>   | Agrega al final                   |
| <code>lista.insert(i,x)</code> | Inserta en posición i             |
| <code>lista.remove(x)</code>   | Elimina primera aparición de x    |
| <code>lista.pop(i)</code>      | Elimina en i (por defecto último) |
| <code>lista.sort()</code>      | Ordena ascendente                 |
| <code>lista.reverse()</code>   | Invierte orden                    |
| <code>lista.index(x)</code>    | Índice de x                       |
| <code>lista.count(x)</code>    | Cuenta apariciones                |

### Métodos de diccionarios

| blue!70!black                      |                               |
|------------------------------------|-------------------------------|
| <code>dict.keys()</code>           | Devuelve las claves           |
| <code>dict.values()</code>         | Devuelve los valores          |
| <code>dict.items()</code>          | Devuelve pares (clave, valor) |
| <code>dict.get(k, default)</code>  | Valor de k o default          |
| <code>dict.setdefault(k, v)</code> | Asigna si no existe           |
| <code>dict.update(otro)</code>     | Fusiona diccionarios          |
| <code>dict.pop(k)</code>           | Elimina y devuelve valor      |

## Métodos de cadenas útiles

|                                               |                                   |
|-----------------------------------------------|-----------------------------------|
| <code>blue!70!black</code>                    |                                   |
| <code>s.upper()</code> / <code>lower()</code> | Mayúsculas / minúsculas           |
| <code>s.capitalize()</code>                   | Primera letra mayúscula           |
| <code>s.title()</code>                        | Primera de cada palabra mayúscula |
| <code>s.strip()</code>                        | Quita espacios extremos           |
| <code>s.replace(old, new)</code>              | Reemplaza subcadena               |
| <code>s.split(sep)</code>                     | Divide en lista                   |
| <code>s.join(lista)</code>                    | Une lista con separador           |
| <code>s.find(sub)</code>                      | Posición de subcadena             |

## Conclusión final

Este documento contiene **todo** el material de las clases, desde los fundamentos hasta proyectos complejos y módulos avanzados. Cada concepto está acompañado de ejemplos reales extraídos de tus propios códigos. No hay mejor forma de aprender que practicando, así que volvé a este PDF cada vez que lo necesites. ¡Seguí programando y nunca pares de aprender!

**Nahuel Villafañe**

Python paso a paso

[github.com/Fisicoloco99](https://github.com/Fisicoloco99)

© 2026 – Licencia Creative Commons

BY-NC-SA 4.0